



COMPSCI 683: Artificial Intelligence

Yair Zick



Inference in First-Order Logic (FOL)

AIMA Chapter 9.1 – 9.3, 9.4.1, 9.5.1 – 9.5.3

Outline

- Reduction to propositional inference
- Unification
- Generalized Modus Ponens
- Forward chaining
- Backward chaining
- Resolution

Converting from First Order to Propositional Logic

- Universal quantifier $\Rightarrow$ propositional rules:
 $\forall x: P(x) \wedge Q(x) \Rightarrow R(x)$ becomes
 $P(a) \wedge Q(a) \Rightarrow R(a)$
 $P(b) \wedge Q(b) \Rightarrow R(b)$
 $\vdots$
- Convert an existential quantifier by adding a new **Skolem constant** (not appearing in KB !)
 $\exists x: P(x)$ becomes $P(x_0)$; the variable x_0 is **new**.

First Order to Propositional Logic

Rules:

$\text{King}(\text{John}),$

$\text{Greedy}(\text{John}),$

$\text{Brother}(\text{Richard}, \text{John}),$

$\forall x: \text{King}(x) \wedge \text{Greedy}(x) \Rightarrow \text{Evil}(x),$

$\exists x: \text{Crown}(x) \wedge \text{Onhead}(x, \text{John}).$

Rules:

$\text{King}(\text{John}),$

$\text{Greedy}(\text{John}),$

$\text{Brother}(\text{Richard}, \text{John}),$

$\text{King}(\text{John}) \wedge \text{Greedy}(\text{John}) \Rightarrow \text{Evil}(\text{John}),$

$\text{King}(\text{Richard}) \wedge \text{Greedy}(\text{Richard}) \Rightarrow \text{Evil}(\text{Richard}),$

$\text{Crown}(C) \wedge \text{Onhead}(C, \text{John}).$

Atomic sentences (e.g. $\text{King}(\text{John}),$
 $\text{Brother}(\text{Richard}, \text{John})$)
are now symbols

Reduction to Propositional Inference

- Every FOL KB can be propositionalized; preserves entailment: α is entailed by new KB iff entailed by original KB
- **Idea:** propositionalize KB and query; infer, return result
- **Problem:** with function symbols, there are infinitely many ground-term substitutions
 - e.g., $x > y \Leftrightarrow (x = next(y)) \vee (\exists z: (x = next(z)) \wedge (z > y))$
 - $x > y$ would convert to
 $x = next(y) \vee x = next(next(y)) \vee x = next(next(next(y))) \dots$

Theorem (Herbrand, 1930). If α is entailed by FOL KB , then it is entailed by a **finite subset** of the propositionalized KB .

- Idea: For $n = 0$ to ∞ do

Does this remind you of any other algorithm?

- create a propositionalized KB_n by instantiating with depth- n terms
- see if α is entailed by this KB_n

What happens if α is not entailed?

Entailment for FOL is **semi-decidable** ($\exists$ algorithms that return TRUE if α is entailed, but no algorithm exists that returns FALSE to every non-entailed α).

Similar to the **halting problem** (Turing, 1936)

Propositionalization is Expensive

- Generates irrelevant things:

$\forall x: \text{King}(x) \wedge \text{Greedy}(x) \Rightarrow \text{Evil}(x)$

$\forall y: \text{Greedy}(y)$

$\text{King}(\textit{John})$

Easily implies that $\text{Evil}(\textit{John})$

... but we will also add the irrelevant

$\text{Greedy}(\textit{Richard})$

- Exponential blowup: a k -ary predicate has d^k instantiations with d constants. We don't need all of them!

Key Idea - Unification

- We can get the inference immediately if we can find a substitution θ such that $\text{King}(x)$ and $\text{Greedy}(y)$ match $\text{King}(\text{John})$ and $\text{Greedy}(\text{John})$

Unifier $\theta = \{x \leftarrow \text{John}, y \leftarrow \text{John}\}$ works

AIMA uses notation
 $x \setminus \text{John}$

- $\text{UNIFY}(p, q)$ outputs a var. substitution θ s.t. $\text{SUBST}(\theta, p) = \text{SUBST}(\theta, q)$

p	q	θ
$\text{Knows}(\text{John}, x)$	$\text{Knows}(\text{John}, \text{Jane})$	$\{x \leftarrow \text{Jane}\}$
$\text{Knows}(\text{John}, x)$	$\text{Knows}(y, \text{Bob})$	$\{x \leftarrow \text{Bob}, y \leftarrow \text{John}\}$
$\text{Knows}(\text{John}, x)$	$\text{Knows}(y, \text{Mother}(y))$	$\{y \leftarrow \text{John}, x \leftarrow \text{Mother}(\text{John})\}$
$\text{Knows}(\text{John}, x)$	$\text{Knows}(x, \text{Bob})$	FAIL

- Standardizing apart** eliminates variable name clashes

Unification – Multiple Unifiers

To unify $\text{Knows}(\text{John}, x)$ and $\text{Knows}(y, z)$,

$\theta = \{y \leftarrow \text{John}, x \leftarrow z\}$  **More General**

or

$\theta = \{y \leftarrow \text{John}, x \leftarrow \text{John}, z \leftarrow \text{John}\}$

There is a unique **most general unifier** (MGU),
up to renaming and substitution of variables

$$MGU = \{y \leftarrow \text{John}, x \leftarrow z\}$$

Finding an MGU: Unification Algorithm

function UNIFY(x, y, θ) **returns** a substitution to make x and y identical

inputs: x , a variable, constant, list, or compound expression

y , a variable, constant, list, or compound expression

θ , the substitution built up so far (optional, defaults to empty)

if $\theta = \text{failure}$ **then return failure**

else if $x = y$ **then return** θ

else if VARIABLE?(x) **then return** UNIFY-VAR(x, y, θ)

else if VARIABLE?(y) **then return** UNIFY-VAR(y, x, θ)

else if COMPOUND?(x) **and** COMPOUND?(y) **then**

return UNIFY(x .ARGS, y .ARGS, UNIFY(x .OP, y .OP, θ))

else if LIST?(x) **and** LIST?(y) **then**

return UNIFY(x .REST, y .REST, UNIFY(x .FIRST, y .FIRST, θ))

else return failure

function UNIFY-VAR(var, x, θ) **returns** a substitution

if $\{var/val\} \in \theta$ **then return** UNIFY(val, x, θ)

else if $\{x/val\} \in \theta$ **then return** UNIFY(var, val, θ)

else if OCCUR-CHECK?(var, x) **then return failure**

else return add $\{var/x\}$ to θ

Unification Algorithm

Given two sentences P, Q , find a variable assignment such that $P = Q$

- Input: P, Q, θ (θ is empty at first)
- If $\theta = \text{FAIL}$ or if $P = Q$ return θ
- If one input is a variable (say, $P = x$), unify a variable with a sentence given θ .
- Otherwise, break up the compound function/list and process their parts recursively.

The UNIFY-VAR function

Given a variable x , a sentence Q , and θ

- If $x \leftarrow val$ (under θ), return $\text{UNIFY}(val, Q, \theta)$; if $Q \leftarrow val$ (under θ), return $\text{UNIFY}(x, val, \theta)$.
- If x appears in Q return FAIL (recursion won't stop!)
- If x does not appear in Q return $\theta \cup \{x \leftarrow Q\}$

It is ok to set x to be a very long sentence:
 $x \leftarrow P(Q(y, z, \ell), A)$

Unification Example

Input: $F(x, y), F(a, z)$

$\text{Unify}(F(x, y), F(a, z), [], [])$

$\text{Unify}([x, y], [a, z], \text{Unify}(F, F, [], []))$

$\text{Unify}([x, y], [a, z], [], [])$

$\text{Unify}(x, a, \text{Unify}(y, z, [], []))$

$\text{Unify}(x, a, [y \leftarrow z]) \Rightarrow$
 $[y \leftarrow z, x \leftarrow a]$



Inference Algorithms for FOL

Generalized Modus Ponens (GMP)

- Original Modus Ponens: $\frac{\alpha_1, \dots, \alpha_k; (\alpha_1 \wedge \dots \wedge \alpha_k \Rightarrow \beta)}{\beta}$
- Generalized (lifted) Modus Ponens:
 - $\text{King}(\text{John}), \forall y: \text{Greedy}(y)$
 - $\forall x: \text{King}(x) \wedge \text{Greedy}(x) \Rightarrow \text{Evil}(x)$
 - **There is some substitution θ** such that the premise and the implication are the same (namely $[x \leftarrow \text{John}, y \leftarrow \text{John}]$)
 - ... we can infer that $\text{Evil}(\text{John})$

Generalized Modus Ponens (GMP)

These sentences have variables $x_1, \dots, x_n$ in their universal quantifiers (the $\forall$ elements)

- We are given a set of FOL sentences $P_1, \dots, P_k$ and an implication rule $R_1 \wedge \dots \wedge R_k \Rightarrow Q$
- If there exists some substitution θ over the variables such that

$$\text{SUBST}(P_1, \theta) = \text{SUBST}(R_1, \theta)$$

$$\text{SUBST}(P_2, \theta) = \text{SUBST}(R_2, \theta)$$

$$\vdots$$

$$\text{SUBST}(P_k, \theta) = \text{SUBST}(R_k, \theta)$$

- ... then $\text{SUBST}(Q; \theta)$ holds!

Soundness of GMP

Assume:

- there exists a substitution θ such that $\text{SUBST}(P_j, \theta) = \text{SUBST}(R_j, \theta)$ for all j .
- $P_1, \dots, P_k$ hold, and that
- $R_1 \wedge \dots \wedge R_k \Rightarrow Q$

We need to prove that $\text{SUBST}(Q, \theta)$ holds as well.

$\exists \theta$ s.t. $\text{SUBST}(P_j, \theta) = \text{SUBST}(R_j, \theta) \ \forall j.$

$P_1, \dots, P_k$ hold, and $R_1 \wedge \dots \wedge R_k \Rightarrow Q$

Lemma: if $P = \forall y_1, \dots, y_\ell: \alpha(y_1, \dots, y_\ell)$ then $P \models \text{SUBST}(P, \theta)$ for any substitution θ

Proof: substitutions are “special cases” of the general rule.

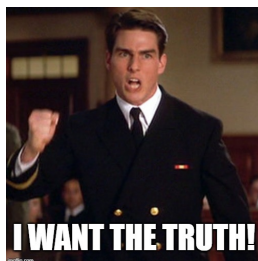
- By Lemma:
 - $\forall j: P_j \models \text{SUBST}(P_j, \theta) = \text{SUBST}(R_j, \theta)$ and
 - $R_1 \wedge \dots \wedge R_k \Rightarrow Q \models$
 $\text{SUBST}(R_1, \theta) \wedge \dots \wedge \text{SUBST}(R_k, \theta) \Rightarrow \text{SUBST}(Q, \theta)$
- We know that
 - $\text{SUBST}(R_1, \theta), \dots, \text{SUBST}(R_k, \theta)$ hold, and that
 - $\text{SUBST}(R_1, \theta) \wedge \dots \wedge \text{SUBST}(R_k, \theta) \Rightarrow \text{SUBST}(Q, \theta)$
- Apply Propositional Logic’s Modus Ponens!

Example KB in FOL

“The law says that it is a crime for an American to sell weapons to hostile nations. The country Nono, an enemy of America, has some missiles, and all of its missiles were sold to it by Colonel West, who is American.”

You may assume that

1. Missiles are weapons
2. Any country that's an enemy of America is a hostile nation



Prove that Colonel West is a criminal...
Using your FOL superpowers

Example KB in FOL

“The law says that **it is a crime for an American to sell weapons to hostile nations**. The country Nono, an enemy of America, has some missiles, and all of its missiles were sold to it by Colonel West, who is American.”

$\forall x, y, z: \text{American}(x) \wedge \text{Weapon}(y) \wedge \text{Hostile}(z) \wedge \text{Sells}(x, y, z) \Rightarrow \text{Criminal}(x)$

Example KB in FOL

“The law says that it is a crime for an American to sell weapons to hostile nations.
The country Nono, an enemy of America,
has some missiles, and all of its missiles were
sold to it by Colonel West, who is American.”

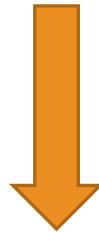
Enemy(Nono, America)

Example KB in FOL

“The law says that it is a crime for an American to sell weapons to hostile nations.

The country Nono, an enemy of America, **has some missiles**, and all of its missiles were sold to it by Colonel West, who is American.”

$\exists x: \text{Owns}(\text{Nono}, x) \wedge \text{Missile}(x)$



**Existential
Instantiation**

$\text{Owns}(\text{Nono}, M_1); \text{Missile}(M_1)$

Example KB in FOL

“The law says that it is a crime for an American to sell weapons to hostile nations. The country Nono, an enemy of America, has some missiles, and **all of its missiles were sold to it by Colonel West**, who is American.”

$\forall x: \text{Missile}(x) \wedge \text{Owns}(\text{Nono}, x) \Rightarrow \text{Sells}(\text{West}, x, \text{Nono})$

Example KB in FOL

“The law says that it is a crime for an American to sell weapons to hostile nations. The country Nono, an enemy of America, has some missiles, and all of its missiles were sold to it by **Colonel West, who is American.**”

American(*West*)

Example KB in FOL

- Missiles are weapons:
$$\forall x: \text{Missile}(x) \Rightarrow \text{Weapon}(x)$$
- Enemies of America are hostile:
$$\forall x: \text{Enemy}(x, \textit{America}) \Rightarrow \text{Hostile}(x)$$

Example KB in FOL

1. $\forall x, y, z: \text{American}(x) \wedge \text{Weapon}(y) \wedge \text{Hostile}(z) \wedge \text{Sells}(x, y, z) \Rightarrow \text{Criminal}(x)$
2. $\text{Enemy}(\text{Nono}, \text{America})$
3. $\text{Owns}(\text{Nono}, M_1)$
4. $\text{Missile}(M_1)$
5. $\forall x: \text{Missile}(x) \wedge \text{Owns}(\text{Nono}, x) \Rightarrow \text{Sells}(\text{West}, x, \text{Nono})$
6. $\text{American}(\text{West})$
7. $\forall x: \text{Missile}(x) \Rightarrow \text{Weapon}(x)$
8. $\forall x: \text{Enemy}(x, \text{America}) \Rightarrow \text{Hostile}(x)$

Example KB in FOL

Want to prove that
Criminal(*West*)

Forward Chaining Algorithm

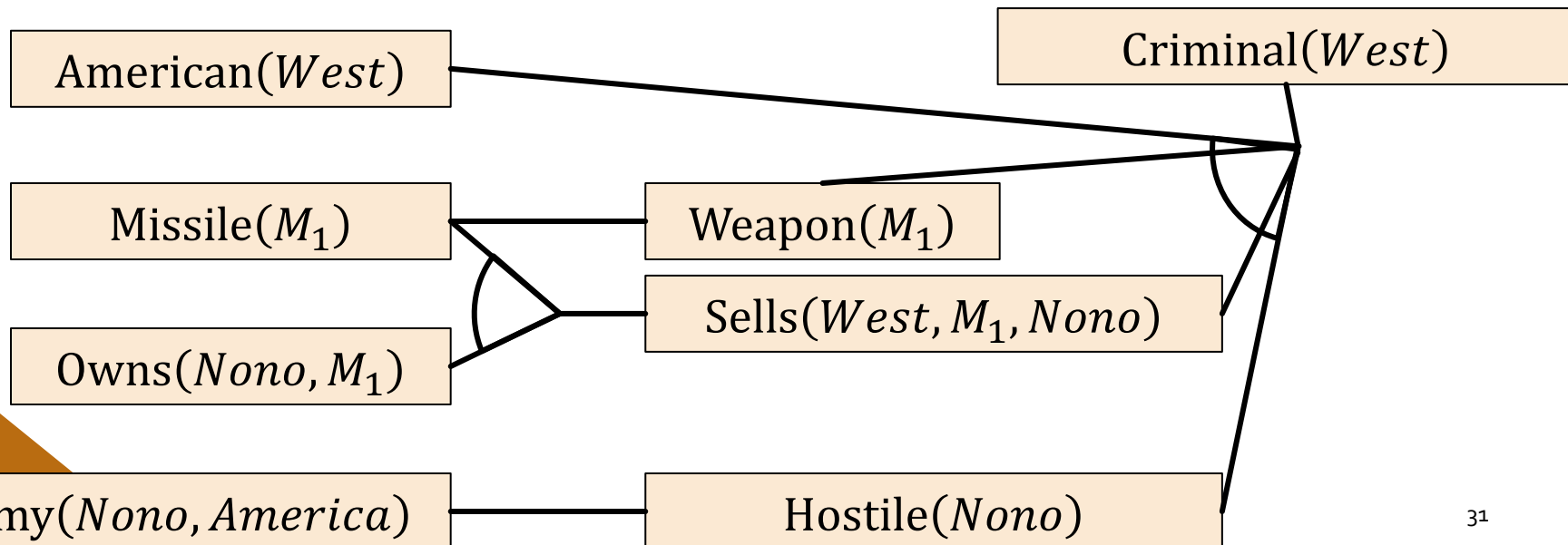
```
function FOL-FC-ASK( $KB, \alpha$ ) returns a substitution or false
  inputs:  $KB$ , the knowledge base, a set of first-order definite clauses
            $\alpha$ , the query, an atomic sentence
  local variables:  $new$ , the new sentences inferred on each iteration

  repeat until  $new$  is empty
     $new \leftarrow \{ \}$ 
    for each  $rule$  in  $KB$  do
       $(p_1 \wedge \dots \wedge p_n \Rightarrow q) \leftarrow \text{STANDARDIZE-VARIABLES}(rule)$ 
      for each  $\theta$  such that  $\text{SUBST}(\theta, p_1 \wedge \dots \wedge p_n) = \text{SUBST}(\theta, p'_1 \wedge \dots \wedge p'_n)$ 
        for some  $p'_1, \dots, p'_n$  in  $KB$ 
         $q' \leftarrow \text{SUBST}(\theta, q)$ 
        if  $q'$  does not unify with some sentence already in  $KB$  or  $new$  then
          add  $q'$  to  $new$ 
           $\phi \leftarrow \text{UNIFY}(q', \alpha)$ 
          if  $\phi$  is not fail then return  $\phi$ 
    add  $new$  to  $KB$ 
  return false
```

Forward Chaining Algorithm

- Input:
 - KB – a knowledge base coded in FOL
 - α – an FOL sentence
 - Output:
 - A variable substitution θ so that $SUBST(KB, \theta) \models \alpha$
 - ... or FALSE if no such θ exists.
-
- At every round, add all newly inferred atomic sentences to KB .
 - Repeat until
 - One of these sentences is α
 - No new sentences can be inferred.

1. $\forall x, y, z: \text{American}(x) \wedge \text{Weapon}(y) \wedge \text{Hostile}(z) \wedge \text{Sells}(x, y, z) \Rightarrow \text{Criminal}(x)$
2. $\text{Enemy}(\text{Nono}, \text{America})$
3. $\text{Owns}(\text{Nono}, M_1)$
4. $\text{Missile}(M_1)$
5. $\forall x: \text{Missile}(x) \wedge \text{Owns}(\text{Nono}, x) \Rightarrow \text{Sells}(\text{West}, x, \text{Nono})$
6. $\text{American}(\text{West})$
7. $\forall x: \text{Missile}(x) \Rightarrow \text{Weapon}(x)$
8. $\forall x: \text{Enemy}(x, \text{America}) \Rightarrow \text{Hostile}(x)$



Properties of Forward Chaining

- Sound and complete for first-order definite clauses
- FC terminates for KB in finite number of iterations if it contains no functions.
- May not terminate in general (i.e., with functions) if α is not entailed
- This is unavoidable: entailment with definite clauses is **semidecidable**

Inefficiencies of Forward Chaining

```
function FOL-FC-ASK( $KB, \alpha$ ) returns a substitution or false
  inputs:  $KB$ , the knowledge base, a set of first-order definite clauses
            $\alpha$ , the query, an atomic sentence
  local variables: new, the new sentences inferred on each iteration

  repeat until new is empty
     $new \leftarrow \{ \}$ 
    for each rule in  $KB$  do
       $(p_1 \wedge \dots \wedge p_n \Rightarrow q) \leftarrow \text{STANDARDIZE-VARIABLES}(\text{rule})$ 
      for each  $\theta$  such that  $\text{SUBST}(\theta, p_1 \wedge \dots \wedge p_n) = \text{SUBST}(\theta, p'_1 \wedge \dots \wedge p'_n)$ 
        for some  $p'_1, \dots, p'_n$  in  $KB$ 
           $q' \leftarrow \text{SUBST}(\theta, q)$ 
          if  $q'$  does not unify with some sentence already in  $KB$  or new then
            add  $q'$  to new
             $\phi \leftarrow \text{UNIFY}(q', \alpha)$ 
            if  $\phi$  is not fail then return  $\phi$ 
    add new to  $KB$ 
  return false
```

Matching rules and known facts is costly

Matching Rule Premises to Known Facts is Costly

Predicate indexing: constant time to retrieve known facts

- $\text{Missile}(x) \Rightarrow \text{Weapon}(x)$: find all facts (e.g., $\text{Missile}(M_1)$) that unify with $\text{Missile}(x)$.

Conjunct ordering problem: apply minimum-remaining-values heuristic of CSP

- e.g., $\text{Missile}(x) \wedge \text{Owns}(\text{Nono}, x) \Rightarrow \text{Sells}(\text{West}, x, \text{Nono})$
if many objects are owned by *Nono* and few missiles, start with $\text{Missile}(x)$ conjunct

Inefficiencies of Forward Chaining

function FOL-FC-ASK(KB, α) **returns** a substitution or *false*
inputs: KB , the knowledge base, a set of first-order definite clauses
 α , the query, an atomic sentence
local variables: new , the new sentences inferred on each iteration

repeat until new is empty
 $new \leftarrow \{ \}$
 for each $rule$ **in** KB **do**
 $(p_1 \wedge \dots \wedge p_n \Rightarrow q) \leftarrow \text{STANDARDIZE-VARIABLES}(rule)$
 for each θ such that $\text{SUBST}(\theta, p_1 \wedge \dots \wedge p_n) = \text{SUBST}(\theta, p'_1 \wedge \dots \wedge p'_n)$
 for some $p'_1, \dots, p'_n$ in KB
 $q' \leftarrow \text{SUBST}(\theta, q)$
 if q' does not unify with some sentence already in KB or new **then**
 add q' to new
 $\phi \leftarrow \text{UNIFY}(q', \alpha)$
 if ϕ is not *fail* **then return** ϕ
 add new to KB
return *false*

Redundant rule matching

Redundant Rule Matchings

Incremental forward chaining: Match rule at time t only if a conjunct in its premise unifies with **new fact** inferred at iteration $t - 1$.

- e.g., $\text{Missile}(x) \Rightarrow \text{Weapon}(x)$ matches against $\text{Missile}(M_1)$ again in 2nd iteration; but, $\text{Weapon}(M_1)$ is already known.

Rete ("Ree-Tee") algorithm:

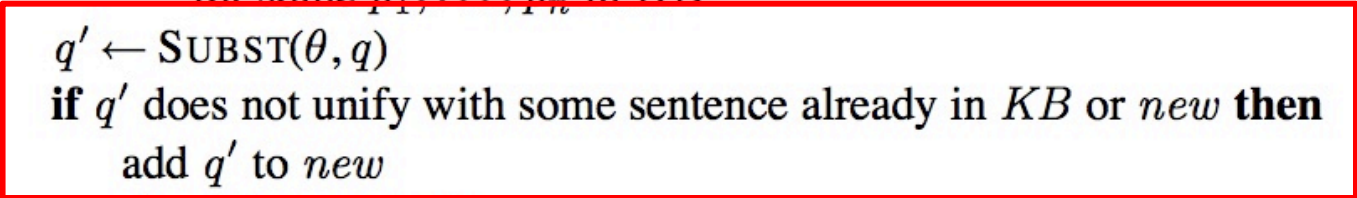
- Don't discard partially matched rules
- Keep track of conjuncts matched against new facts; avoid duplicate work:

$\text{American}(x) \wedge \text{Weapon}(y) \wedge \text{Sells}(x, y, z) \wedge \text{Hostile}(z) \Rightarrow \text{Criminal}(x)$ is partially matched against $\text{American}(West)$ in first iteration.

Inefficiencies of Forward Chaining

```
function FOL-FC-ASK( $KB, \alpha$ ) returns a substitution or false
  inputs:  $KB$ , the knowledge base, a set of first-order definite clauses
            $\alpha$ , the query, an atomic sentence
  local variables: new, the new sentences inferred on each iteration

  repeat until new is empty
     $new \leftarrow \{ \}$ 
    for each rule in  $KB$  do
       $(p_1 \wedge \dots \wedge p_n \Rightarrow q) \leftarrow \text{STANDARDIZE-VARIABLES}(\text{rule})$ 
      for each  $\theta$  such that  $\text{SUBST}(\theta, p_1 \wedge \dots \wedge p_n) = \text{SUBST}(\theta, p'_1 \wedge \dots \wedge p'_n)$ 
        for some  $p'_1, \dots, p'_n$  in  $KB$ 
           $q' \leftarrow \text{SUBST}(\theta, q)$ 
          if  $q'$  does not unify with some sentence already in  $KB$  or new then
            add  $q'$  to new
             $\phi \leftarrow \text{UNIFY}(q', \alpha)$ 
            if  $\phi$  is not fail then return  $\phi$ 
    add new to  $KB$ 
  return false
```



Generating Irrelevant Facts

Backward Chaining Algorithm

```
function FOL-BC-ASK(KB, query) returns a generator  
return FOL-BC-OR(KB, query, { })
```

At least one of the rules in the *KB* needs to imply *goal*

```
generator FOL-BC-OR(KB, goal,  $\theta$ ) yields a substitution  
for each rule (lhs  $\Rightarrow$  rhs) in FETCH-RULES-FOR-GOAL(KB, goal) do  
  (lhs, rhs)  $\leftarrow$  STANDARDIZE-VARIABLES((lhs, rhs))  
  for each  $\theta'$  in FOL-BC-AND(KB, lhs, UNIFY(rhs, goal,  $\theta$ )) do  
    yield  $\theta'$ 
```

```
generator FOL-BC-AND(KB, goals,  $\theta$ ) yields a substitution  
if  $\theta = \text{failure}$  then return  
else if LENGTH(goals) = 0 then yield  $\theta$   
else do
```

All elements of *goals* need to be true

```
  first, rest  $\leftarrow$  FIRST(goals), REST(goals)  
  for each  $\theta'$  in FOL-BC-OR(KB, SUBST( $\theta$ , first),  $\theta$ ) do  
    for each  $\theta''$  in FOL-BC-AND(KB, rest,  $\theta'$ ) do  
      yield  $\theta''$ 
```

Backward Chaining: Proof Tree

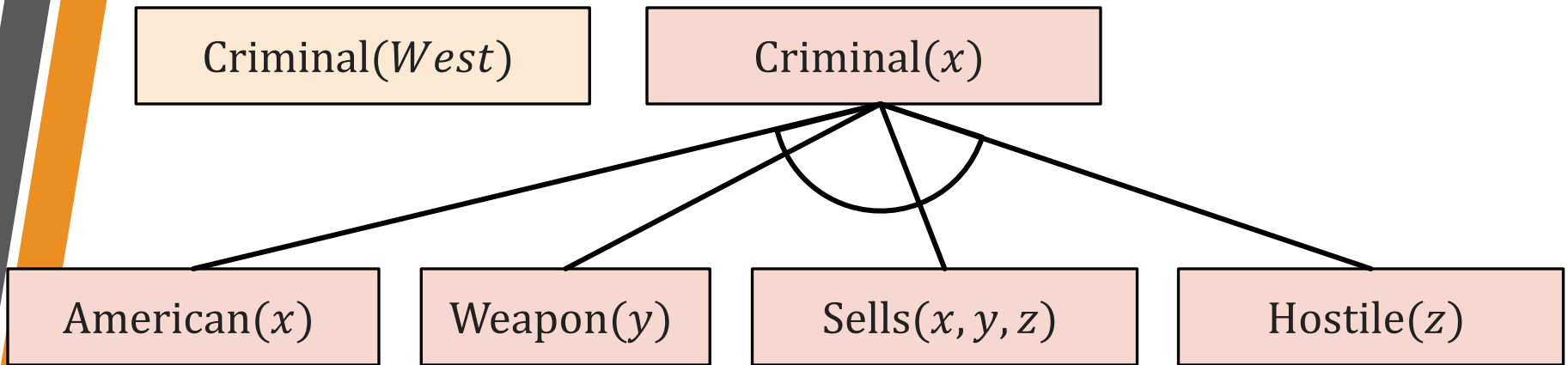
Criminal(*West*)

Call FOL-BC-OR on *KB* and Criminal(*West*)

FETCH-RULE on Criminal(*West*):

$\forall x, y, z: \text{American}(x) \wedge \text{Weapon}(y)$
 $\wedge \text{Hostile}(z) \wedge \text{Sells}(x, y, z) \Rightarrow \text{Criminal}(x)$

Backward Chaining: Proof Tree

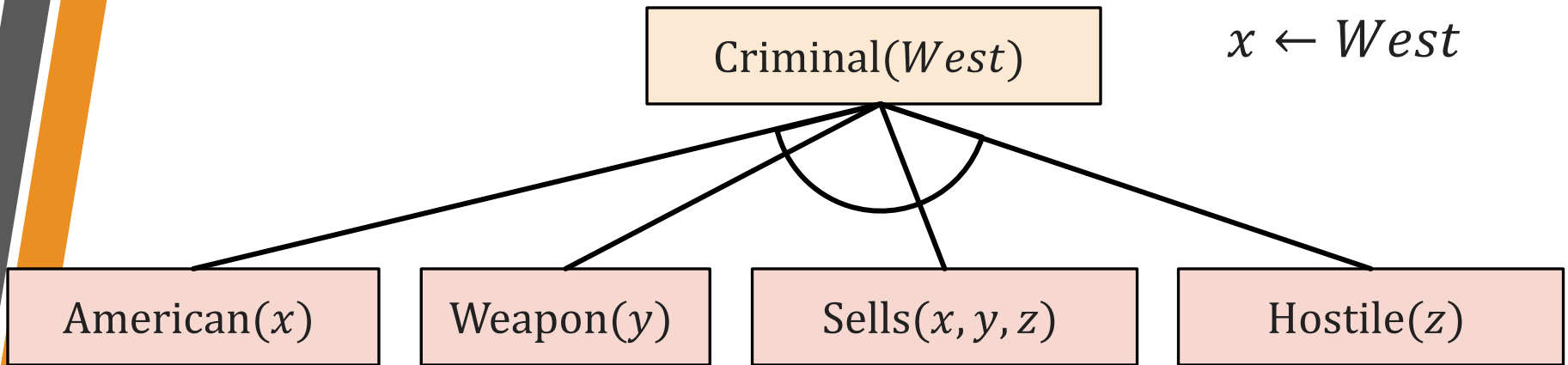


Unify

$American(x) \wedge Weapon(y) \wedge Hostile(z)$
 $\wedge Sells(x, y, z) \Rightarrow Criminal(x)$

and $Criminal(West)$, result in $\theta = [x \leftarrow West]$

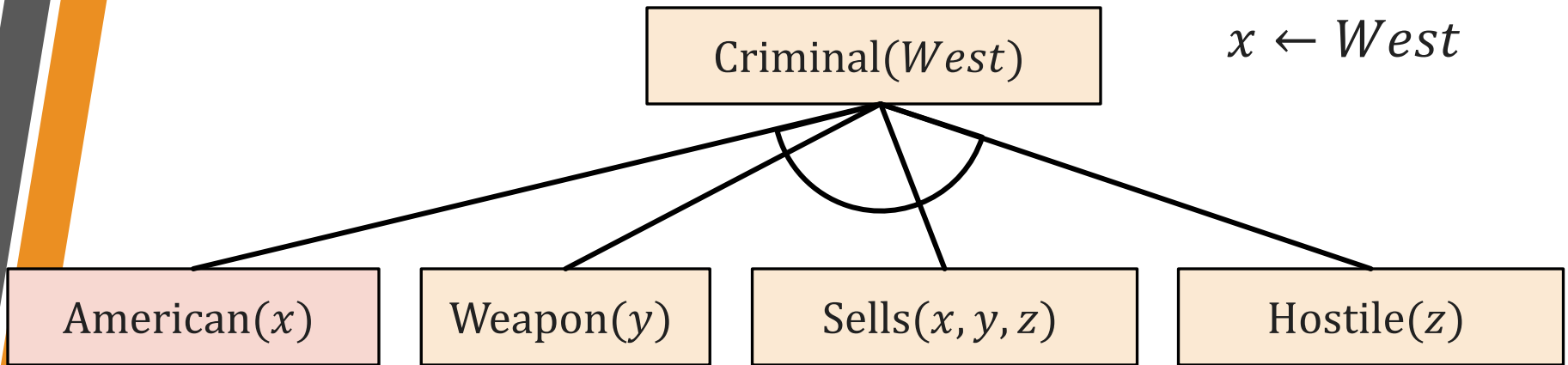
Backward Chaining: Proof Tree



Call FOL-BC-AND on the list of rules

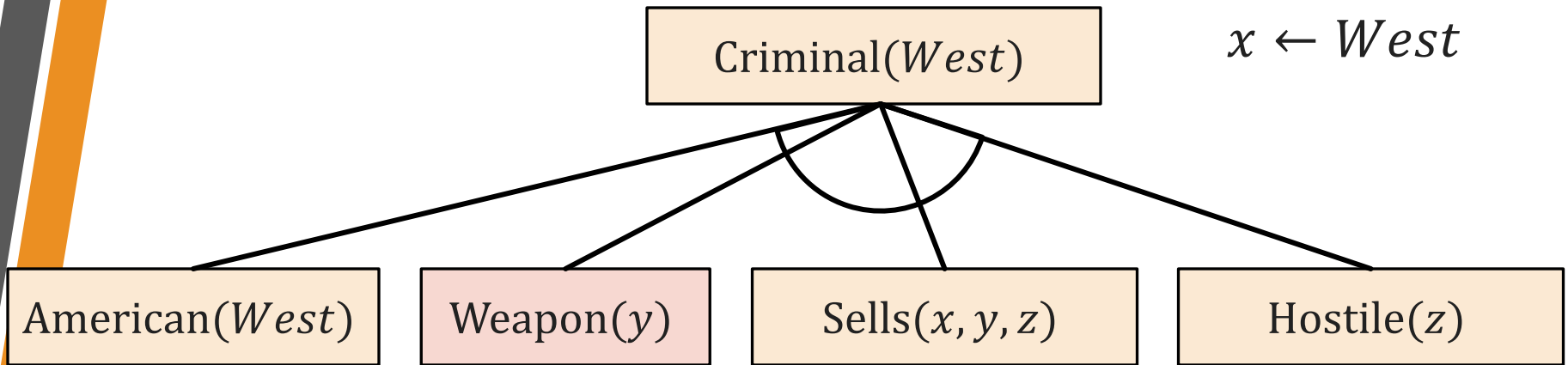
$[\text{American}(x), \text{Weapon}(y), \text{Sells}(x, y, z), \text{Hostile}(z)]$

Backward Chaining: Proof Tree



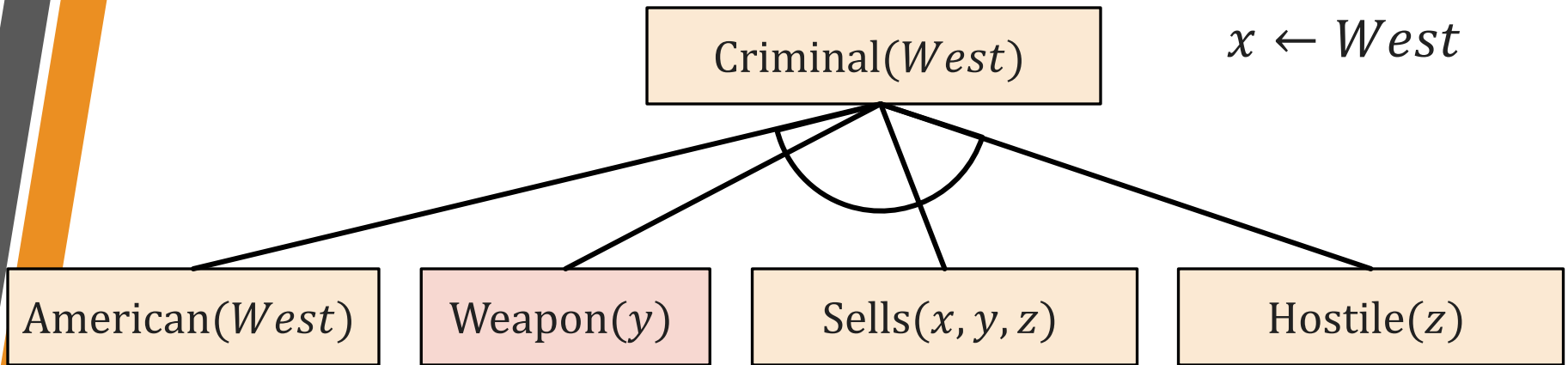
American(x) gets unified with American(*West*) in the *KB* with existing substitution

Backward Chaining: Proof Tree



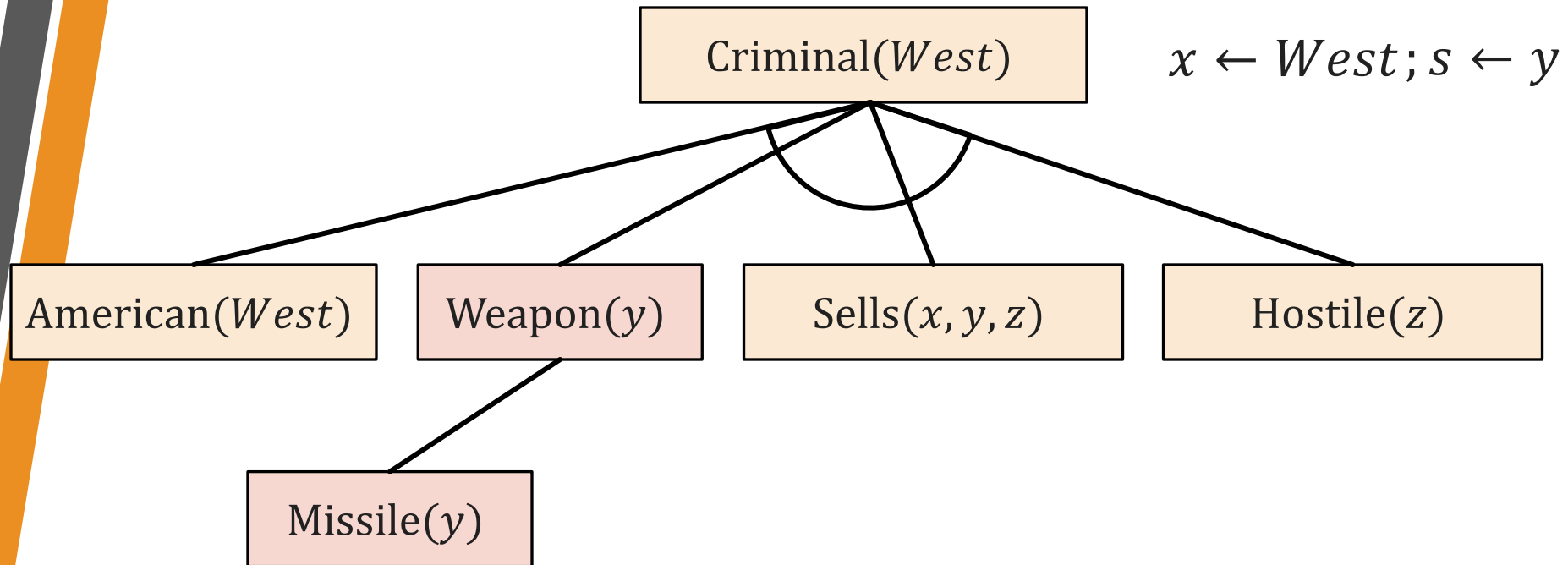
American(x) gets unified with
American(*West*) in the *KB* with existing
substitution

Backward Chaining: Proof Tree



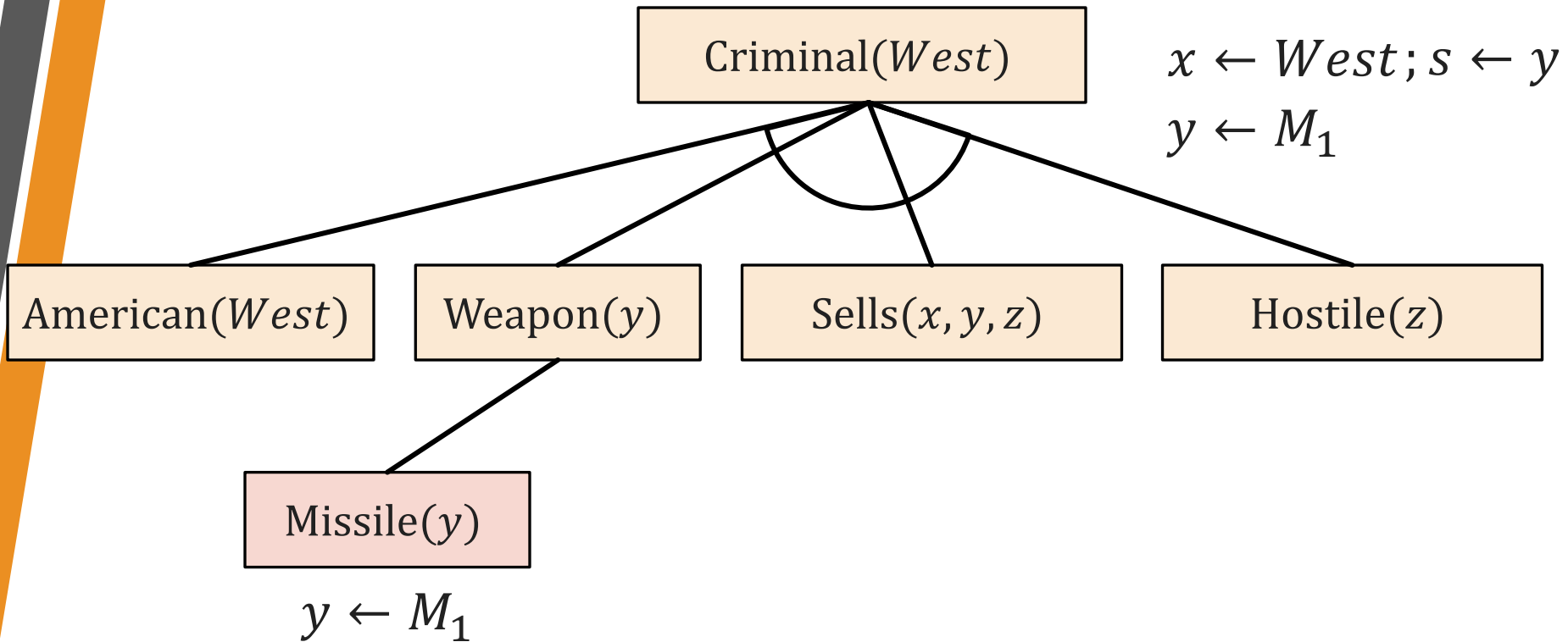
We call FOL-BC-OR on $Weapon(y)$ and fetch the rule $Missile(s) \Rightarrow Weapon(s)$

Backward Chaining: Proof Tree



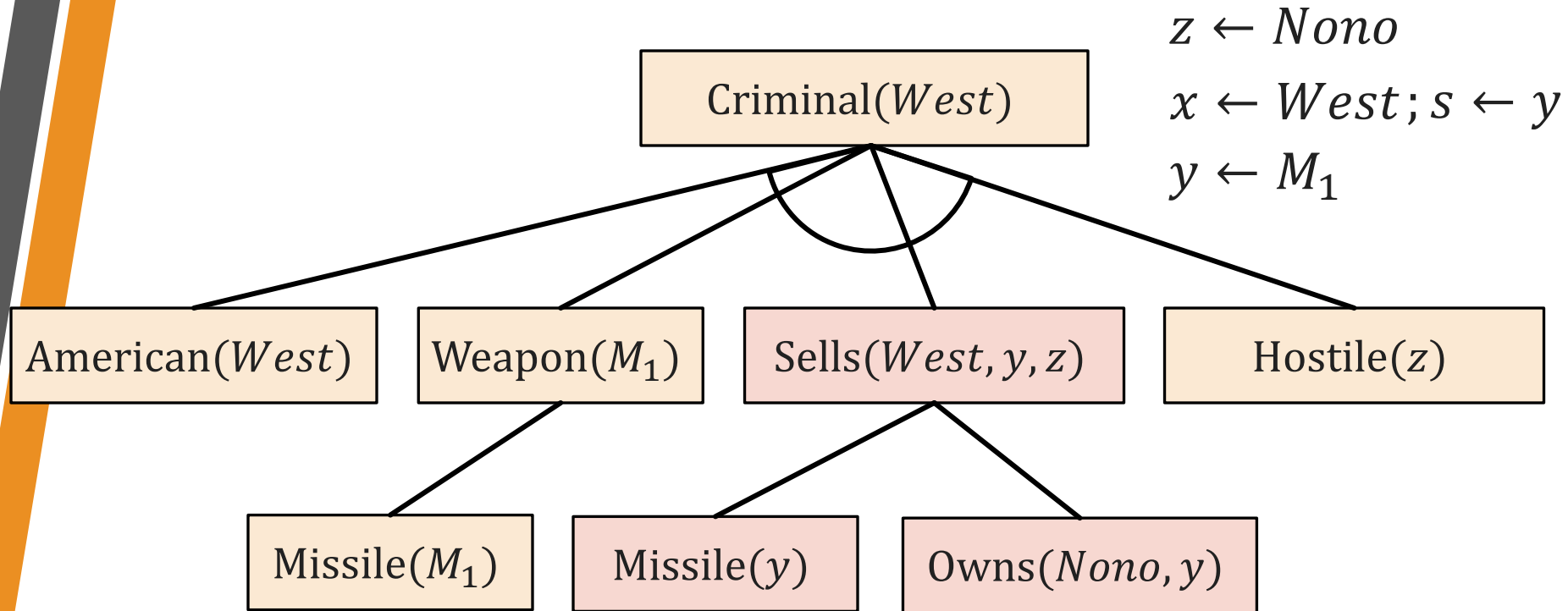
Unifying we get $[s \leftarrow y]$

Backward Chaining: Proof Tree



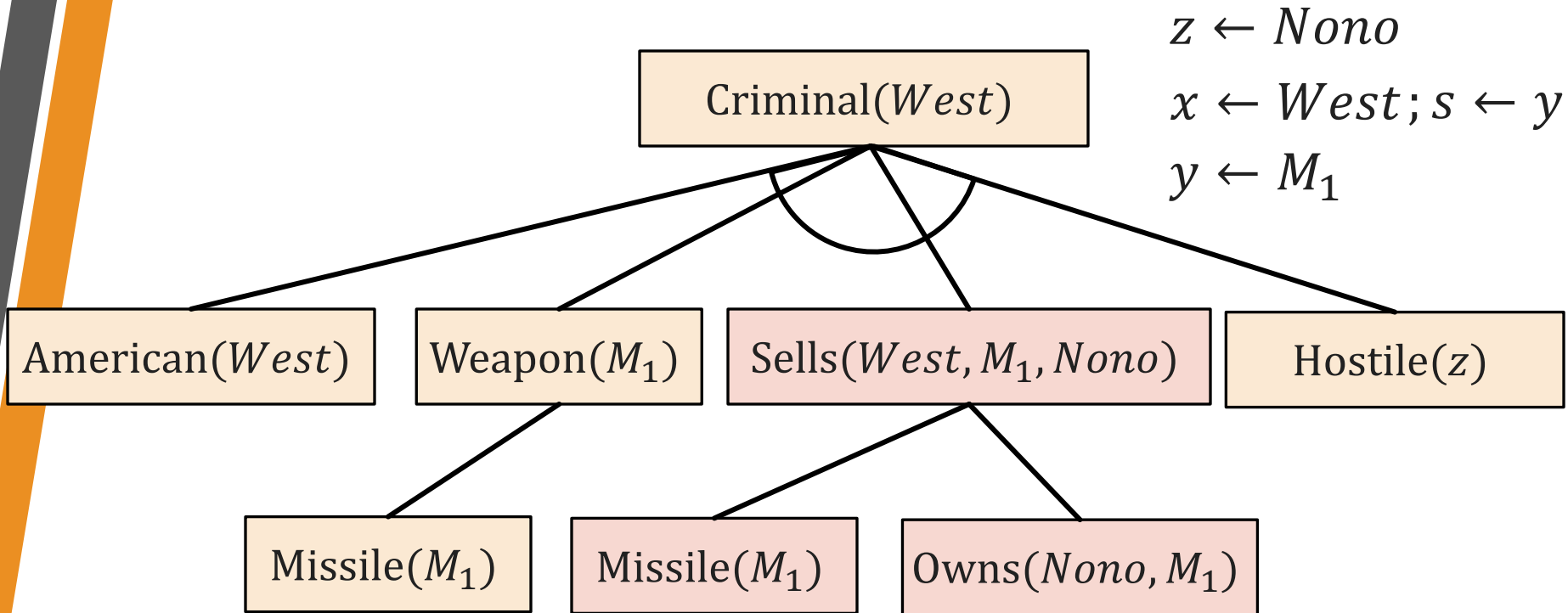
Unify on Missile(M_1) and Missile(*y*) outputs
 $y \leftarrow M_1$.

Backward Chaining: Proof Tree



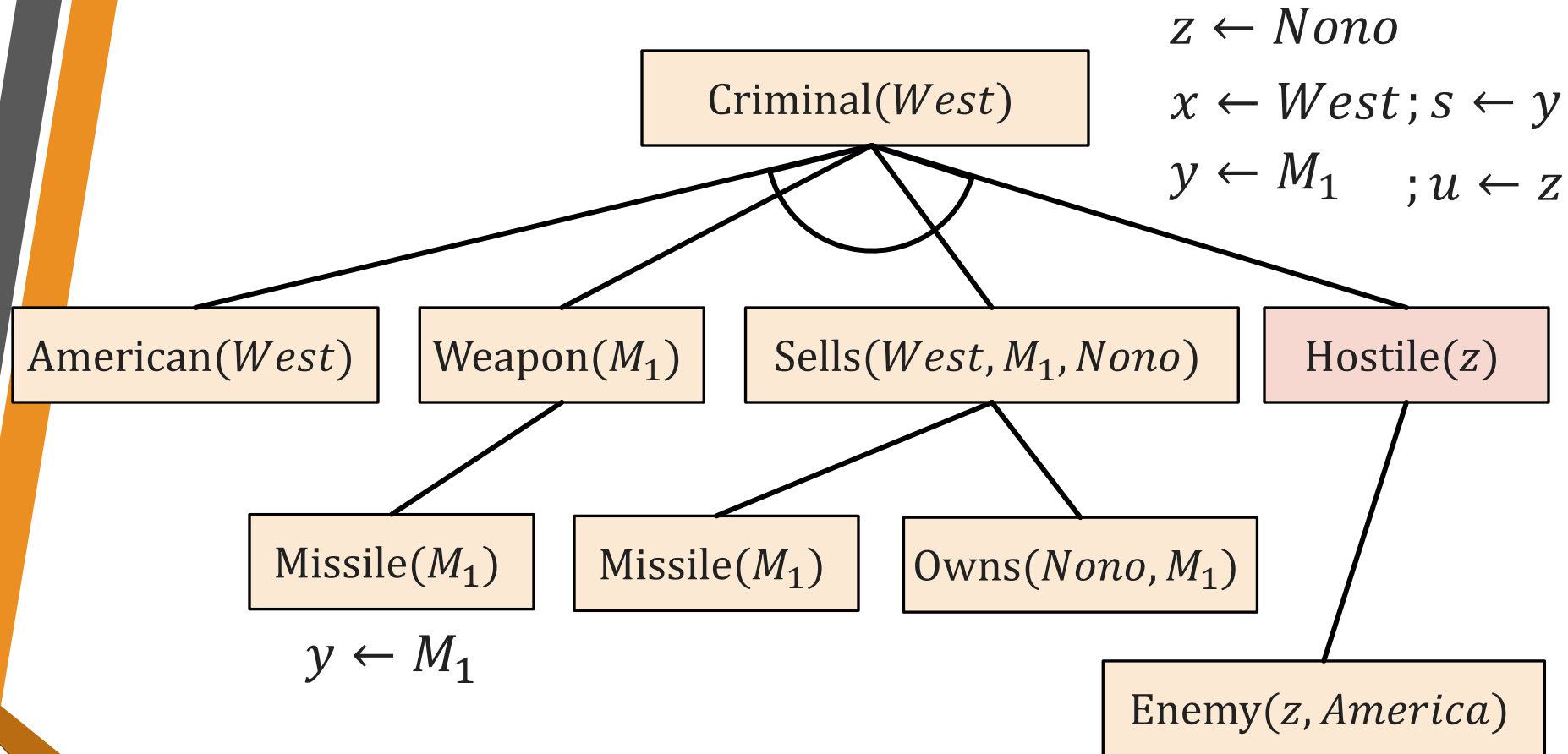
Fetch $\text{Missile}(t) \wedge \text{Owns}(\text{Nono}, t) \Rightarrow \text{Sells}(\text{West}, t, \text{Nono})$
UNIFY to get $t \leftarrow y$ and $z \leftarrow \text{Nono}$

Backward Chaining: Proof Tree



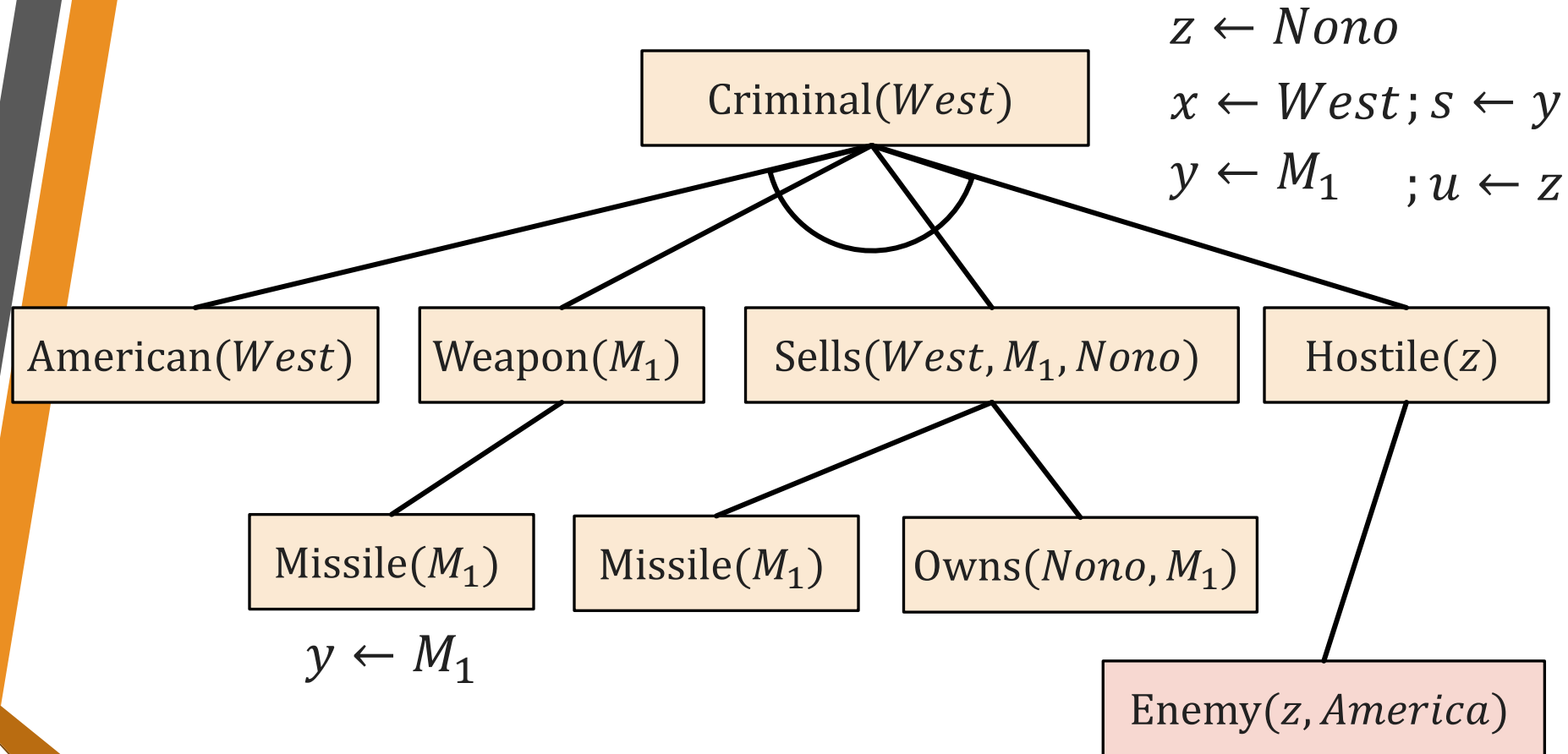
no contradiction with $y \leftarrow M_1$ so we are good!

Backward Chaining: Proof Tree



Fetch $\text{Enemy}(u, \text{America}) \Rightarrow \text{Hostile}(u)$
 UNIFY to get $u \leftarrow z$

Backward Chaining: Proof Tree



UNIFY with $\text{Enemy}(\text{Nono}, \text{America})$, get no contradiction with current assignment to z .

Properties of Backward Chaining

- Depth-first search: space is linear in size of proof
- Incomplete due to infinite loops: fix by checking current goal against every goal on stack
- Inefficient due to repeated subgoals (both success and failure): fix by caching solutions to previous subgoals
- Widely used for **logic programming**

Resolution in FOL: Convert to CNF

“Everyone who loves all animals is loved by someone”

$$\forall x: (\forall y: \text{Animal}(y) \Rightarrow \text{Loves}(x, y)) \Rightarrow \exists y: \text{Loves}(y, x)$$

- **Eliminate implications: $A \Rightarrow B$ becomes $\neg A \vee B$**

$$\begin{aligned} \forall x: & \neg(\forall y: \neg \text{Animal}(y) \vee \text{Loves}(x, y)) \\ & \vee \exists y: \text{Loves}(y, x) \end{aligned}$$

- **De Morgan's rule:**

$$\neg \forall x: P \equiv \exists x: \neg P \text{ and } \neg \exists x: P \equiv \forall x: \neg P$$

$$\forall x: \left(\exists y: \neg(\neg \text{Animal}(y) \vee \text{Loves}(x, y)) \right) \vee \exists y: \text{Loves}(y, x)$$

$$\forall x: \left(\exists y: \neg \neg \text{Animal}(y) \wedge \neg \text{Loves}(x, y) \right) \vee \exists y: \text{Loves}(y, x)$$

$$\forall x: \left(\exists y: \text{Animal}(y) \wedge \neg \text{Loves}(x, y) \right) \vee \exists y: \text{Loves}(y, x)$$

Resolution in FOL: Convert to CNF

“Everyone who loves all animals is loved by someone”

$$\forall x: (\forall y: \text{Animal}(y) \Rightarrow \text{Loves}(x, y)) \Rightarrow \exists y: \text{Loves}(y, x)$$

- **Standardize variables:**

$$\forall x: (\exists y: \text{Animal}(y) \wedge \neg \text{Loves}(x, y)) \vee \exists z: \text{Loves}(z, x)$$

- **Skolemize existential quantifiers:**

replace with functions **depending on external universal quantifier**. Cannot just apply existential instantiation (y and z may depend on x)!

$$\forall x: (\text{Animal}(F(x)) \wedge \neg \text{Loves}(x, F(x))) \vee \text{Loves}(G(x), x)$$

$F(x)$ and $G(x)$ are called Skolem Functions.

Resolution in FOL: Convert to CNF

“Everyone who loves all animals is loved by someone”

$$\forall x: (\forall y: \text{Animal}(y) \Rightarrow \text{Loves}(x, y)) \Rightarrow \exists z: \text{Loves}(z, x)$$

- **Drop Universal Quantifiers:**

$$\left(\text{Animal}(F(x)) \wedge \neg \text{Loves}(x, F(x)) \right) \vee \text{Loves}(G(x), x)$$

- **Distribute $\vee$ over $\wedge$:**

$$(A \vee B) \wedge C \equiv (A \wedge C) \vee (B \wedge C)$$

- **Distribute $\wedge$ over $\vee$:**

$$(A \wedge B) \vee C \equiv (A \vee C) \wedge (B \vee C)$$

$$\left(\text{Animal}(F(x)) \vee \text{Loves}(G(x), x) \right) \wedge \left(\neg \text{Loves}(x, F(x)) \vee \text{Loves}(G(x), x) \right)$$

CNF Resolution in FOL

- FOL literals are **complements** if one unifies with negation of the other. $F(x, y), \neg F(u, G(z))$ are complements with $\theta = [x \leftarrow u, y \leftarrow G(z)]$
- Suppose that a, b are complements that can be unified with θ , then the two clauses $(\ell_1 \vee \dots \vee \ell_q \vee a)$ and $(m_1 \vee \dots \vee m_r \vee b)$ resolve to
$$SUBST(\theta, \ell_1 \vee \dots \vee \ell_q \vee m_1 \vee \dots \vee m_r)$$

The clauses are **standardized apart**: share no variables

- For example,

$$\frac{\neg \text{Loves}(u, v) \vee \neg \text{Kills}(u, v); \text{Animal}(F(x)) \vee \text{Loves}(G(x), x)}{\neg \text{Kills}(G(x), x) \vee \text{Animal}(F(x))}$$

where $\text{UNIFY}(\neg \text{Loves}(u, v), \neg \text{Loves}(G(x), x)) = [u \leftarrow G(x), v \leftarrow x]$

CNF Resolution in FOL

- **First-order factoring:** removes redundant literals by reducing 2 literals to one if they are unifiable.

- For example,

$(P(x) \vee G(a, b)) ; (\neg P(y) \vee G(k, \ell))$ results in $G(a, b) \vee G(k, \ell)$.

- Apply factoring to get $G(k, \ell)$
- To prove $KB \models \alpha$, show that $KB \wedge \neg\alpha$ results in a contradiction.

Example KB in FOL

1. $\neg \text{American}(x) \vee \neg \text{Weapon}(y) \vee \neg \text{Hostile}(z) \vee \neg \text{Sells}(x, y, z) \vee \text{Criminal}(x)$
2. $\text{Enemy}(\text{Nono}, \text{America})$
3. $\text{Owns}(\text{Nono}, M_1) \wedge \text{Missile}(M_1)$
4. $\neg \text{Missile}(q) \vee \neg \text{Owns}(\text{Nono}, q) \vee \text{Sells}(\text{West}, q, \text{Nono})$
5. $\text{American}(\text{West})$
6. $\neg \text{Missile}(r) \vee \text{Weapon}(r)$
7. $\neg \text{Enemy}(s, \text{America}) \vee \text{Hostile}(s)$

Query: $\alpha = \text{Criminal}(\text{West})$



Add: $\neg \alpha = \neg \text{Criminal}(\text{West})$ to KB, find contradiction

$\neg \text{American}(x) \vee \neg \text{Weapon}(y) \vee \neg \text{Hostile}(z) \vee \neg \text{Sells}(x, y, z) \vee \text{Criminal}(x)$

$\neg \text{Criminal}(\text{West})$

$x \leftarrow \text{West}$

$\text{American}(\text{West})$

$\neg \text{American}(\text{West}) \vee \neg \text{Weapon}(y) \vee \neg \text{Hostile}(z) \vee \neg \text{Sells}(\text{West}, y, z)$

$\neg \text{Missile}(r) \vee \text{Weapon}(r)$

$\neg \text{Weapon}(y) \vee \neg \text{Hostile}(z) \vee \neg \text{Sells}(\text{West}, y, z)$

$r \leftarrow y$

$\text{Missile}(M_1)$

$\neg \text{Missile}(y) \vee \neg \text{Hostile}(z) \vee \neg \text{Sells}(\text{West}, y, z)$

$y \leftarrow M_1$

$\neg \text{Missile}(q) \vee \neg \text{Owns}(\text{Nono}, q) \vee \text{Sells}(\text{West}, q, \text{Nono})$

$\neg \text{Hostile}(z) \vee \neg \text{Sells}(\text{West}, M_1, z)$

$q \leftarrow M_1$
 $z \leftarrow \text{Nono}$

$\text{Missile}(M_1)$

$\neg \text{Hostile}(\text{Nono}) \vee \neg \text{Missile}(M_1) \vee \neg \text{Owns}(\text{Nono}, M_1)$

$\text{Owns}(\text{Nono}, M_1)$

$\neg \text{Hostile}(\text{Nono}) \vee \neg \text{Owns}(\text{Nono}, M_1)$

$\neg \text{Enemy}(s, \text{America}) \vee \text{Hostile}(s)$

$\neg \text{Hostile}(\text{Nono})$

$s \leftarrow \text{Nono}$

$\text{Enemy}(\text{Nono}, \text{America})$

$\neg \text{Enemy}(\text{Nono}, \text{America})$